

SoC RISC-V com Acelerador AES-128

Relatório Final - 3ª Fase Embarcotech

Felipe Marinho de Carvalho

Brasília, 22 de fevereiro de 2026

Resumo

Este documento apresenta o projeto de um System-on-Chip (SoC) baseado no processador RISC-V PicoRV32, integrado a um acelerador criptográfico AES-128. O sistema foi desenvolvido em Verilog, sintetizado para uma Colorlight-i9 utilizando ferramentas open-source (Yosys, NextPNR). O relatório descreve a arquitetura de hardware, o mapeamento de memória, o desenvolvimento do software de controle, a metodologia de síntese e os resultados de verificação. O objetivo principal é descarregar o processamento criptográfico da CPU, permitindo encriptação rápida de blocos de dados recebidos via serial.

Conteúdo

1	Introdução	4
1.1	Motivação	4
1.2	Objetivos	4
1.2.1	Objetivo Geral	4
1.2.2	Objetivos Específicos	5
1.3	Disponibilidade do Código	5
2	Fundamentação Teórica	6
2.1	Arquitetura RISC-V e PicoRV32	6
2.2	Padrão Criptográfico AES-128	6
2.3	Comunicação Serial	6
2.4	Entrada e Saída Mapeada em Memória	7
3	Arquitetura de Hardware	8
3.1	Visão da Arquitetura	8
3.2	Topologia do Sistema e Controle de Barramento	8
3.3	Mapeamento de Memória e Decodificação	9
3.4	Interface do Acelerador AES-128	10
3.5	Interface do Controlador UART	10
4	Desenvolvimento de Software	12
4.1	Ambiente de Compilação e Toolchain	12
4.2	Mapeamento de Memória em C	13
4.3	Drivers de Controle da UART	13
4.4	Fluxo de Execução e Controle do AES	14
5	Metodologia de Síntese e Implementação Física	16
5.1	Fluxo de Ferramentas Open-Source	16
6	Verificação e Resultados	17
6.1	Simulação (Testbench)	17
6.2	Ocupação de Recursos e Síntese	17
6.3	Validação Funcional em Hardware	18
6.4	Análise de Desempenho	19
7	Conclusão	21
7.1	Resultados Obtidos	21

Lista de Figuras

3.1	Diagrama de blocos lógicos da arquitetura SoC	9
6.1	Formas de onda da simulação em nível RTL (GTKWave)	17

Listings

3.1	Lógica de decodificação e multiplexação do barramento	9
4.1	Arquivo de ligação (sections.lds)	12
4.2	Instanciação e inicialização da memória RAM em hardware	12
4.3	Cabeçalho de mapeamento de registradores de hardware (soc_map.h) .	13
4.4	Drivers de comunicação serial	13
4.5	Firmware principal do SoC (firmware.c)	14
6.1	Script de validação física em Python	18

Capítulo 1

Introdução

A criptografia é uma peça fundamental na era digital, protegendo informações em praticamente todas as áreas da tecnologia — desde pequenos dispositivos da Internet das Coisas (IoT) com recursos limitados até grandes servidores em nuvem. Para garantir a confidencialidade e a integridade dos dados, o uso de algoritmos criptográficos robustos é indispensável.

No entanto, quando algoritmos complexos são executados puramente em software, eles costumam gerar gargalos de desempenho significativos. O algoritmo *Advanced Encryption Standard* (AES-128), por exemplo, exige a manipulação constante de matrizes, substituições de bytes e deslocamentos de bits. Quando um processador de propósito geral tenta resolver essas operações passo a passo, de forma estritamente sequencial, ele consome uma quantidade excessiva de ciclos de clock, reduzindo a eficiência do sistema.

1.1 Motivação

A motivação deste trabalho consiste em explorar a prática de *co-design* (projeto conjunto) de hardware e software para mitigar as limitações da execução puramente sequencial. A abordagem propõe a criação de um *System-on-Chip* (SoC) construído em torno de um processador da arquitetura RISC-V.

A proposta de arquitetura do projeto baseia-se no princípio da aceleração por meio de circuito dedicado: em vez de forçar a CPU a calcular a criptografia através de centenas de instruções de software, o processamento matemático do AES-128 é transferido para um circuito dedicado. Ao transferir essa carga de trabalho para um acelerador de hardware projetado especificamente para este fim, a encriptação é resolvida em uma fração do tempo. Isso reduz a latência da operação, como também libera o processador para gerenciar outras tarefas do sistema.

1.2 Objetivos

1.2.1 Objetivo Geral

Projetar, implementar e validar um sistema embarcado baseado na arquitetura RISC-V integrado a um acelerador criptográfico de hardware para o algoritmo AES-128, visando a otimização do tempo de execução através do *co-design* de hardware e software.

1.2.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes passos metodológicos:

1. **Integração da Arquitetura:** Instanciar o núcleo do RISC-V (PicoRV32) e mapear o coprocessador AES-128, estabelecendo a comunicação entre eles através de um barramento de memória (MMIO - *Memory-Mapped I/O*).
2. **Desenvolvimento de Firmware:** Criar o código em linguagem C responsável por controlar o envio de chaves e dados em texto a ser criptografado para o acelerador, bem como a coleta do texto cifrado.
3. **Síntese e Implementação Física:** Utilizar um fluxo de ferramentas de síntese *open-source* (Yosys, NextPNR e openFPGALoader) para transformar código em Verilog em um circuito real e gravar ele em uma FPGA da família Lattice ECP5.
4. **Validação:** Desenvolver uma interface de comunicação serial (UART) e um *script* em Python para injetar vetores de teste para validar a criptografia.

1.3 Disponibilidade do Código

O repositório pode ser acessado através do GitHub no seguinte endereço: <https://github.com/felipe-mcarvalho/Embarcatech-3Fase>

Capítulo 2

Fundamentação Teórica

2.1 Arquitetura RISC-V e PicoRV32

O RISC-V é uma arquitetura de conjunto de instruções (ISA) de código aberto baseada nos princípios de computação com conjunto reduzido de instruções (RISC). Diferente de arquiteturas proprietárias e fechadas do mercado, o RISC-V permite que desenvolvedores criem, modifiquem e implementem seus próprios processadores livremente.

Neste projeto, utilizou-se o PicoRV32, uma implementação específica da arquitetura RISC-V escrita em Verilog. O PicoRV32 foi projetado para ser uma CPU extremamente leve, otimizada para ocupar a menor área de silício possível. Ele atua como o cérebro do *System-on-Chip*, gerenciando o fluxo de controle geral do sistema.

2.2 Padrão Criptográfico AES-128

O *Advanced Encryption Standard* (AES) é um algoritmo de criptografia simétrica adotado para a proteção de informações confidenciais. Na sua variante AES-128, o algoritmo opera agrupando os dados em blocos lógicos de 128 bits e aplicando uma chave criptográfica do mesmo tamanho.

O processo de cifragem ocorre iterativamente ao longo de múltiplas rodadas (*rounds*). Cada um desses ciclos executa quatro transformações matemáticas principais: a substituição de bytes (*SubBytes*), o deslocamento de linhas da matriz (*ShiftRows*), a mistura de colunas (*MixColumns*) e a adição da chave específica daquela rodada (*AddRound-Key*). Como essas etapas envolvem operações com matrizes, elas consomem uma quantidade grande de tempo e de ciclos de máquina quando executadas passo a passo por um processador comum. Isso justifica a necessidade de transferir essas operações para um circuito de hardware dedicado.

2.3 Comunicação Serial

A interface UART (*Universal Asynchronous Receiver-Transmitter*) é um dos protocolos de comunicação mais tradicionais para a troca de dados entre dispositivos. Sua transmissão é classificada como assíncrona, o que significa que não há um fio de *clock* compartilhado entre o computador e a placa FPGA; em vez disso, ambos os lados concordam previamente com uma velocidade de transmissão fixa (o *baud rate*).

Como os dados podem chegar pela via serial em momentos imprevisíveis e o processador pode estar momentaneamente ocupado resolvendo outras tarefas, torna-se necessário o uso de uma estrutura de dados do tipo FIFO (*First-In, First-Out*). Implementada como um *buffer* circular na memória, a FIFO armazena os bytes recebidos em uma fila, garantindo que o processador possa lê-los assim que estiver livre, evitando a perda de informações durante a transmissão.

2.4 Entrada e Saída Mapeada em Memória

O *Memory-Mapped I/O* (MMIO) é uma técnica de arquitetura de computadores que permite ao processador se comunicar com periféricos externos usando exatamente as mesmas instruções que usaria para ler ou gravar dados na memória RAM.

Sob a ótica do processador, não há diferença entre salvar o valor de uma variável em um programa ou enviar um comando de disparo para o acelerador criptográfico: ambos são vistos apenas como endereços de memória. Em nível de *hardware*, existe um circuito decodificador de endereços que intercepta as requisições da CPU e direciona os sinais para o periférico correto com base na faixa de endereço solicitada.

Capítulo 3

Arquitetura de Hardware

3.1 Visão da Arquitetura

A arquitetura desenvolvida neste projeto baseia-se no conceito de *System-on-Chip* (SoC). Nele, um núcleo de processamento central é responsável por organizar o fluxo de dados entre diferentes periféricos e aceleradores de hardware dedicados. Este capítulo detalha a topologia do sistema, os mecanismos de controle no barramento e como o módulo criptográfico AES-128 foi integrado.

3.2 Topologia do Sistema e Controle de Barramento

O elemento central do sistema é o processador PicoRV32, que implementa o conjunto básico de instruções da arquitetura RISC-V (RV32IM). Na estrutura implementada, o processador atua como mestre único, controlando um barramento unificado de dados e endereços de 32 bits. Os outros blocos do projeto, como a memória RAM, a UART e o núcleo AES, operam apenas como escravos, respondendo às requisições de leitura ou escrita iniciadas pela CPU.

A comunicação no barramento é controlada por um protocolo baseado nos sinais `mem_valid` e `mem_ready`. Quando o processador precisa fazer uma transação, ele ativa o sinal `mem_valid` e envia o endereço no barramento `mem_addr`. A transação fica em espera até que o periférico de destino sinalize que a operação terminou ativando o sinal `mem_ready`.

A Figura 3.1 ilustra como esses componentes estão interligados no projeto.

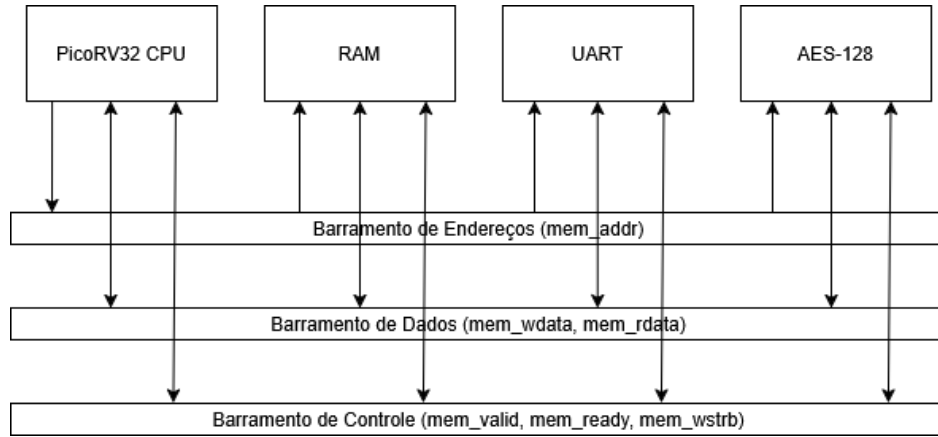


Figura 3.1: Diagrama de blocos lógicos da arquitetura SoC

3.3 Mapeamento de Memória e Decodificação

Para que o processador RISC-V pudesse interagir com seus periféricos, utilizou-se a técnica de I/O Mapeado em Memória (*Memory-Mapped I/O* - MMIO). Nessa abordagem, os registradores de controle e dados de todos os dispositivos escravos recebem endereços como se fossem espaços na memória principal. O sistema foi dividido utilizando os 8 bits mais significativos (MSB) do barramento para selecionar cada módulo, como mostrado na Tabela 3.1.

Tabela 3.1: Mapeamento de memória da arquitetura (MMIO)

Endereço Base	Periférico	Função no Sistema
0x00000000	RAM interna	Armazena as instruções e a pilha (<i>Firmware</i>)
0x02000000	Controlador UART	Interface serial assíncrona (TX/RX)
0x04000000	Acelerador AES	Coprocessador criptográfico de hardware

A lógica está no módulo principal (`soc_top.v`), decodificador de endereços avalia a parte superior do barramento (`mem_addr[31:24]`) para gerar sinais de seleção exclusivos para cada periférico.

Para a leitura de dados, a arquitetura usa um multiplexador. Como todos os periféricos compartilham o mesmo fio para devolver dados à CPU (`mem_rdata`), os sinais de seleção descritos na Listagem 3.1 garantem que apenas o componente escolhido envie sinais para o barramento.

```

1  wire sel_ram  = (mem_addr[31:24] == 8'h00);
2  wire sel_uart = (mem_addr[31:24] == 8'h02);
3  wire sel_aes  = (mem_addr[31:24] == 8'h04);
4
5  // Mux Principal: Decide qual periferico preenche a via de retorno
6  // da CPU
7  reg [31:0] rdata_ram;
8  assign mem_rdata =
9      sel_ram ? rdata_ram : // Acesso a RAM
10     sel_uart ? (mem_addr[2] ? {30'b0, w_tx_active, w_fifo_empty} :
11                 {24'b0, w_fifo_data_out}) : // UART
12     sel_aes  ? aes_rdata_val : // Acelerador AES

```

```
11 32'b0; // Default
```

Listing 3.1: Lógica de decodificação e multiplexação do barramento

3.4 Interface do Acelerador AES-128

O algoritmo do AES opera com blocos de dados e chaves de 128 bits de tamanho, enquanto o barramento do RISC-V é limitado a 32 bits. Por conta dessa diferença de tamanho, foi necessário criar uma técnica para carregar os dados em partes.

Para resolver isso, criou-se um banco de registradores internos a partir do endereço base do acelerador (0x04000000). A Tabela 3.2 mostra como esses registradores foram organizados.

Tabela 3.2: Mapa de registradores internos da interface AES-128

Endereço Relativo	Nome do Registrador	Descrição Funcional
0x00 a 0x0C	AES_DIN_x	4 palavras (32 bits) para receber o texto a ser criptografado.
0x10 a 0x1C	AES_KEY_x	4 palavras (32 bits) para receber a Chave Criptográfica.
0x20	AES_CTRL_REG	Registrador de controle. A escrita no bit 0 gera o pulso de <i>Start</i> .
0x30 a 0x3C	AES_DOUT_x	4 palavras (32 bits) contendo o Texto Cifrado gerado.
0x40	AES_STATUS_REG	Registrador de estado. A leitura do bit 0 retorna o sinal <i>Done</i> .

Essa estrutura permite que o *firmware* envie os dados em partes, transferindo blocos de 32 bits até montar os 128 bits do *Plaintext* e da *Key*. O controle do acelerador é feito apenas pelo registrador de controle. Ao escrever o bit menos significativo (LSB) no endereço 0x20, a CPU aciona o pulso `aes_start`, iniciando o módulo dedicado para a operação de criptografia.

Enquanto a execução acontece, o sinal de que o núcleo AES terminou (`aes_done`) é conectado ao registrador de estado (`AES_STATUS`). Isso permite que o *software* implemente um mecanismo de verificação contínua, detectando o fim dos 11 ciclos (*rounds*) do processo antes de receber o texto cifrado.

3.5 Interface do Controlador UART

Assim como o acelerador AES, a UART foi mapeada em memória para que a CPU pudesse interagir com ele através de operações simples de leitura e escrita. O módulo foi alocado no endereço base 0x02000000 e dividido em dois registradores, conforme demonstrado na Tabela 3.3.

Tabela 3.3: Mapa de registradores internos da interface UART

Endereço Relativo	Nome do Registrador	Descrição Funcional
0x00	UART_DATA_REG	Registrador bidirecional (8 bits). A escrita envia um byte para transmissão (TX). A leitura extrai o byte mais antigo da fila de recepção (FIFO RX).
0x04	UART_STATUS_REG	Registrador de estado (Somente Leitura). O Bit 0 (RX_EMPTY) indica se a FIFO está vazia. O Bit 1 (TX_BUSY) indica se a via de transmissão está ocupada.

A implementação física dessa separação foi resolvida diretamente no multiplexador de leitura do sistema, apresentado anteriormente na Listagem 3.1.

A instrução Verilog avalia o sinal `mem_addr[2]` como seleta de um sub-multiplexador interno. Se o bit for verdadeiro (1), o hardware concatena as *flags* de estado (`w_tx_active` e `w_fifo_empty`) com 30 bits de zeros, entregando o *status* no barramento. Caso seja falso (0), o hardware devolve os 8 bits do dado extraído da FIFO (`w_fifo_data_out`) preenchidos com 24 zeros. Essa abordagem permite que a decodificação seja feita em um único ciclo de *clock*.

Capítulo 4

Desenvolvimento de Software

Para controlar o fluxo de dados entre o computador externo e o módulo criptográfico de hardware (AES-128), foi desenvolvido um *firmware* dedicado em linguagem C. Como o sistema embarcado projetado não possui um Sistema Operacional para intermediar os processos, o software interage diretamente e fisicamente com os registradores do hardware. Este capítulo detalha a cadeia de ferramentas de compilação utilizada, a construção da camada de abstração de hardware e a lógica de controle principal implementada para o processador RISC-V.

4.1 Ambiente de Compilação e Toolchain

A compilação do código-fonte para a arquitetura alvo foi realizada utilizando a *toolchain* cruzada GNU GCC específica para a família RISC-V de 32 bits (`riscv32-unknown-elf-gcc`).

O mapeamento do executável gerado para a memória RAM física do SoC, iniciando no endereço base `0x00000000`, foi gerado por um *Linker Script* (`sections.lds`). Em conjunto com este script de ligação, um código de inicialização (`start.S`) foi encarregado de configurar o ponteiro de pilha (*Stack Pointer*) nos primeiros ciclos de clock, preparando o ambiente de execução antes de desviar o fluxo do processador para a função `main()` em C, conforme demonstrado na Listagem 4.1.

```
1 MEMORY
2 {
3     ram (rwx) : ORIGIN = 0x00000000, LENGTH = 0x1000 /* 4KB */
4 }
```

Listing 4.1: Arquivo de ligação (`sections.lds`)

O tamanho da memória RAM principal foi definido em 4 Kilobytes. No código de *hardware* (Verilog), isso equivale a instanciar uma matriz de 1024 posições de 32 bits. Essa capacidade foi escolhida para não gastar muitos recursos físicos da FPGA para a criação de memória.

Como o projeto não possui uma memória externa para carregar o sistema quando a placa é ligada, o programa compilado (`firmware.hex`) precisou ser embutido diretamente no próprio *hardware*. Isso foi feito durante a síntese, utilizando o comando `$readmemh`, conforme demonstrado na Listagem 4.2.

```
1 // Matriz de 4KB (1024 palavras de 32 bits)
2 reg [31:0] memory [0:1023];
```

```

3
4 initial begin
5     // Carrega o firmware em C compilado para dentro do hardware
6     $readmemh("firmware.hex", memory);
7 end

```

Listing 4.2: Instanciação e inicialização da memória RAM em hardware

4.2 Mapeamento de Memória em C

A conexão entre a linguagem C e os sinais elétricos do barramento de hardware foi estabelecida através da técnica de Entrada e Saída Mapeada em Memória (*Memory-Mapped I/O* - MMIO).

Esta abstração foi consolidada no arquivo de cabeçalho `soc_map.h`, apresentado na Listagem 4.3. Este arquivo define os endereços base de cada periférico escravo no barramento e implementa macros utilizando ponteiros.

```

1 // --- Enderecos Base (Memory Mapped I/O) ---
2 #define RAM_BASE_ADDR    0x00000000
3 #define UART_BASE_ADDR   0x02000000
4 #define AES_BASE_ADDR    0x04000000
5
6 // --- Registradores da UART ---
7 #define UART_DATA_REG     (*(volatile uint32_t*)(UART_BASE_ADDR + 0x00))
8 #define UART_STATUS_REG  (*(volatile uint32_t*)(UART_BASE_ADDR + 0x04))
9
10 // Mascaras de Bits da UART
11 #define UART_RX_EMPTY    0x01 // Bit 0: FIFO Vazia
12 #define UART_TX_BUSY     0x02 // Bit 1: Transmissor Ocupado

```

Listing 4.3: Cabeçalho de mapeamento de registradores de hardware (`soc_map.h`)

4.3 Drivers de Controle da UART

O controle da comunicação serial bidirecional foi implementado em nível de driver através da verificação contínua dos registradores de *status* da UART, conforme detalhado na Listagem 4.4.

```

1 void uart_putc(char c) {
2     // Aguarda enquanto o transmissor estiver ocupado
3     while (UART_STATUS_REG & UART_TX_BUSY);
4     UART_DATA_REG = c;
5 }
6
7 char uart_getc() {
8     // Aguarda ate que chegue um dado (FIFO nao vazia)
9     while (UART_STATUS_REG & UART_RX_EMPTY);
10    return (char)UART_DATA_REG;
11 }

```

Listing 4.4: Drivers de comunicação serial

Estas funções atuam de forma bloqueante, utilizando as máscaras de bits (*Bit-masks*) definidas no cabeçalho. O processador aloca seu processamento em laços de repetição (`while`) até que a fila (FIFO) de recepção pegue novos dados, ou até que a via de transmissão esteja livre. Esta abordagem garante a integridade da transmissão, assegurando que nenhum byte seja sobrescrito no registrador de dados ou perdido durante o trânsito elétrico pelo cabo.

4.4 Fluxo de Execução e Controle do AES

A função principal do sistema (`main`) atua como a rotina central de controle. Sua responsabilidade é: a captura de dados externos, o transporte destes ao núcleo criptográfico e a devolução do resultado processado, conforme ilustrado na Listagem 4.5.

```

1  int main() {
2      // 1. Configuracao da Chave Criptografica
3      AES_KEY_3 = 0x6d696e68;
4      AES_KEY_2 = 0x615f6368;
5      AES_KEY_1 = 0x6176655f;
6      AES_KEY_0 = 0x31323334;
7
8      while(1) {
9          // 2. Recepcão e Montagem do Plaintext via UART
10         for (int i = 3; i >= 0; i--) {
11             uint32_t word = 0;
12             word |= (uint32_t)uart_getc() << 24;
13             word |= (uint32_t)uart_getc() << 16;
14             word |= (uint32_t)uart_getc() << 8;
15             word |= (uint32_t)uart_getc();
16             (*(volatile uint32_t*)(AES_BASE_ADDR + (i * 4))) = word;
17         }
18
19         uint32_t t_start = read_cycle(); // Captura de ciclos
20
21         // 3. Disparo da Execucao no Hardware (Start)
22         AES_CTRL_REG = AES_START_BIT;
23         AES_CTRL_REG = 0;
24
25         // 4. Sincronizacao e Espera (Polling)
26         while (!(AES_STATUS_REG & AES_DONE_BIT));
27
28         uint32_t t_end = read_cycle();
29         uint32_t cycles_taken = t_end - t_start;
30
31         // 5. Extracao do Ciphertext e Transmissao
32         for (int i = 3; i >= 0; i--) {
33             uint32_t res = (*(volatile uint32_t*)(AES_BASE_ADDR + 0x30
34 + (i * 4)));
35             uart_putc((char)(res >> 24));
36             uart_putc((char)(res >> 16));
37             uart_putc((char)(res >> 8));
38             uart_putc((char)(res));
39         }
40
41         // Transmissao do tempo de hardware gasto
42         uart_putc((char)(cycles_taken >> 24));

```



```

42     uart_putc((char)(cycles_taken >> 16));
43     uart_putc((char)(cycles_taken >> 8));
44     uart_putc((char)(cycles_taken));
45 }
46 return 0;
47 }

```

Listing 4.5: Firmware principal do SoC (firmware.c)

O fluxo de controle é estruturado em etapas sequenciais. Inicialmente, a chave criptográfica de 128 bits é gravada diretamente nos registradores base do acelerador AES. Em seguida, o sistema entra em um laço infinito onde a CPU atua recebendo dezesseis bytes isolados via porta serial. Através de operações lógicas de deslocamento (*Shift Left*) e a operação ou (*OR*), estes bytes são concatenados para formar palavras completas de 32 bits, que preenchem os registradores de entrada do acelerador.

Com os dados nas posições corretas e o instante exato de *clock* salvo para análise de desempenho, o processador gera um pulso lógico escrevendo o bit de nível alto seguido de nível baixo no registrador de controle (AES_CTRL_REG). Este pulso dispara a máquina de estados independente do coprocessador AES.

Durante o cálculo criptográfico, a CPU entra em um estado de espera, monitorando ininterruptamente a flag de prontidão (AES_DONE_BIT). Assim que o hardware finaliza a cifragem, o laço de espera é interrompido, o tempo gasto é calculado subtraindo os ciclos de clock atuais, e os 128 bits resultantes são lidos do hardware e serializados de volta para o computador junto dos dados de latência.

Capítulo 5

Metodologia de Síntese e Implementação Física

Após a validação comportamental da arquitetura em nível de transferência de registradores (RTL) através de simulação computacional, a etapa seguinte do co-design consistiu em implementar o modelo lógico para o hardware físico. Este projeto foi implementado tendo como alvo uma FPGA da família Lattice ECP5, especificamente o componente LFE5U-45F. Este chip encontra-se embarcado na placa de desenvolvimento Colorlight i9.

5.1 Fluxo de Ferramentas Open-Source

Para a síntese e implementação física do circuito, optou-se pela de código aberto (*open-source*), disponibilizados pelo pacote OSS CAD Suite. O processo de conversão do código Verilog em portas lógicas configuráveis foi dividido em quatro estágios.

Inicialmente, a ferramenta **Yosys** atuou no processo de síntese lógica. O compilador analisou a hierarquia do projeto a partir do módulo de topo, inferindo os componentes lógicos e traduzindo o código Verilog para uma *netlist* (lista de conexões) composta por portas primitivas e *Look-Up Tables* (LUTs) específicas da arquitetura ECP5.

O arquivo resultante foi submetido à ferramenta **NextPNR**, responsável pelas etapas de Posicionamento e Roteamento (*Place and Route*). O NextPNR mapeou a *netlist* lógica para os recursos físicos exatos da malha da FPGA.

A etapa final foi feita pelo **ecppack**, que converteu a configuração física mapeada em um arquivo (*bitstream*). Por fim, a transferência deste *bitstream* para a memória da placa Colorlight foi executada através da ferramenta de gravação **openFPGALoader**.

Capítulo 6

Verificação e Resultados

6.1 Simulação (Testbench)

Antes da implementação física na FPGA, o comportamento lógico do sistema foi validado em ambiente de simulação de hardware utilizando a ferramenta Icarus Verilog. O testbench desenvolvido (*soc_tb.v*) instanciou o módulo de topo do projeto com os sinais de clock e reset, além de gerar os dados seriais da interface UART. A Figura 6.1 ilustra as formas de onda exportadas durante a simulação, demonstrando visualmente o sinal *aes_done* subindo depois de 11 ciclos de clock em relação ao sinal *aes_start*

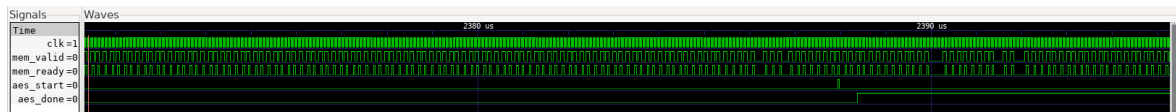


Figura 6.1: Formas de onda da simulação em nível RTL (GTKWave)

6.2 Ocupação de Recursos e Síntese

O código foi traduzido e mapeado para as unidades físicas da FPGA Lattice ECP5, especificamente o modelo LFE5U-45F. Conforme os dados extraídos pelo roteador e sumarizados na Tabela 6.1, o sistema completo consumiu 6.899 blocos lógicos combinacionais (TRELLIS_COMB), o que representa apenas 15% da capacidade total da FPGA. O número de registradores (TRELLIS_FF) alocados foi de 1.292 unidades, equivalente a um consumo de apenas 2% do disponível na arquitetura.

A memória RAM de 4 Kilobytes, instanciada para abrigar o *firmware*, foi mapeada para 2 blocos de memória (EBR - DP16KD).

Tabela 6.1: Ocupação de recursos físicos na FPGA Lattice ECP5-45F

Recurso Físico	Utilizado	Disponível	Ocupação (%)
TRELLIS_COMB	6.899	43.848	15%
TRELLIS_FF	1.292	43.848	2%
TRELLIS_IO	13	245	5%
DP16KD	2	108	1%
TRELLIS_RAMW	32	5.481	0%
DCCA	1	56	1%

6.3 Validação Funcional em Hardware

Após a gravação do *bitstream* na FPGA, foi realizada o teste na placa física. Para isso, desenvolveu-se um script na linguagem Python, executado no computador, responsável por injetar o texto a ser criptografado (*plaintext*) via porta serial e a coleta do texto cifrado (*ciphertext*) devolvido pelo hardware. A Listagem 6.1 apresenta o código de validação.

```

1 import serial
2 import time
3 import sys
4 import struct
5
6 PORT = 'COM6' # Verifique a porta no gerenciador de dispositivos
7 BAUD = 115200
8 TIMEOUT_SEC = 2.0
9 CLOCK_FREQ_HZ = 25000000 # 25 MHz
10
11 PLAINTEXT = b"projeto_riscv_26"
12 EXPECTED_CIPHERTEXT_HEX = "66a9df27b2cfbd97e0a5ad31770b01fd"
13
14 def print_header():
15     print("=" * 70)
16     print("    Validacao e Analise de tempo: SoC RISC-V + AES-128")
17     print("=" * 70)
18
19 def main():
20     print_header()
21     try:
22         ser = serial.Serial(PORT, baudrate=BAUD, timeout=TIMEOUT_SEC)
23     except Exception as e:
24         print(f"[ERRO] {e}")
25         sys.exit(1)
26
27     ser.reset_input_buffer()
28     ser.reset_output_buffer()
29
30     print(" Iniciando Transacao...")
31     t_start_pc = time.perf_counter()
32
33     ser.write(PLAINTEXT)
34
35     # Leitura de 20 bytes (16 Ciphertext + 4 informacoes de tempo
    geradas pelo Hardware)

```

```

36     resposta = ser.read(20)
37
38     t_end_pc = time.perf_counter()
39     ser.close()
40
41     if len(resposta) != 20:
42         print(f"[FAIL] Timeout! Esperados 20 bytes, recebidos {len(
43             resposta)}.")
44         sys.exit(1)
45
46     # Extraindo dados
47     ciphertext_bytes = resposta[:16]
48     ciclos_bytes = resposta[16:20]
49
50     # Converte os 4 bytes do timestamp para um inteiro (Big Endian)
51     ciclos_hw = struct.unpack('>I', ciclos_bytes)[0]
52
53     # Calculos de Analise Temporal
54     total_ms = (t_end_pc - t_start_pc) * 1000
55     tempo_interno_aes_ms = (ciclos_hw / CLOCK_FREQ_HZ) * 1000
56     tempo_interno_aes_us = tempo_interno_aes_ms * 1000
57     tempo_uart_ms = total_ms - tempo_interno_aes_ms
58
59     received_hex = ciphertext_bytes.hex()
60
61     print(f"[VERIFICATION] Resultado Obtido : {received_hex}")
62     if received_hex == EXPECTED_CIPHERTEXT_HEX:
63         print("[VERIFICATION] STATUS : [PASS] - Exato!")
64     else:
65         print("[VERIFICATION] STATUS : [FAIL] -
66             Incompatibilidade!")
67
68     print("-" * 70)
69     print("    ANALISE DE DESEMPENHO")
70     print("-" * 70)
71     print(f" Ciclos de Clock no SoC : {ciclos_hw} ciclos")
72     print(f" Tempo Interno (SoC + AES) : {tempo_interno_aes_us:.2f}
73         us ({tempo_interno_aes_ms:.6f} ms)")
74     print(f" Tempo de Transmissao (UART) : {tempo_uart_ms:.2f} ms")
75     print(f" Tempo Total : {total_ms:.2f} ms")
76     print("=" * 70)
77
78 if __name__ == '__main__':
79     main()

```

Listing 6.1: Script de validação física em Python

6.4 Análise de Desempenho

A análise de desempenho teve como objetivo quantificar o tempo real de execução do co-processador criptográfico. Para medir o tempo exato de processamento em hardware, utilizou-se o contador de ciclos interno do processador PicoRV32. O firmware foi configurado para ler este registrador em dois momentos: imediatamente antes do disparo do sinal de início do núcleo AES e logo após a sinalização de conclusão da

criptografia. Os resultados na placa física revelaram que o núcleo AES, operando em conjunto com o processador, consumiu um total de 33 ciclos de clock para encriptar um bloco de 128 bits. Considerando a frequência de 25 MHz, cujo período equivale a 40 nanossegundos, o tempo ficou em 1,32 μ s.

Tabela 6.2: Resumo do tempo da transação

Parâmetro	Valor Obtido
Frequência do Clock	25 MHz
Ciclos de Processamento (SoC + AES)	33 ciclos
Tempo de Execução Interno	1,32 μ s
Taxa de Transmissão Serial (UART)	115200 bps
Tempo Total da Transação	3,79 ms

Em contrapartida, o tempo total da transação foi medido pelo computador através do script de validação. O tempo total registrado, desde o envio do texto a ser criptografado até a recepção do texto cifrado via interface serial, fixou-se em 3,79 milissegundos. Ao contrastar os tempos apresentados na Tabela 6.2, fica evidente que a maior parte da latência da transação é gerada de forma exclusiva pela comunicação através do protocolo UART operando a 115200 bits por segundo, e não pelo cálculo matemático, provando a eficiência do acelerador.

Capítulo 7

Conclusão

7.1 Resultados Obtidos

O projeto atendeu aos requisitos propostos. A integração entre o *firmware bare-metal* executado no processador RISC-V e o coprocessador criptográfico AES-128 operou com sucesso através do barramento MMIO. A validação física na FPGA comprovou a exatidão do módulo criptográfico na geração do texto criptografado. A medição do tempo de execução revelou que o cálculo em hardware consumiu 33 ciclos de clock (1,32 μ s). Embora não tenha sido desenvolvida uma versão do algoritmo rodando em software na CPU para comparação direta, o baixo número de ciclos mostra que a transferência da carga de trabalho para o hardware dedicado é eficiente.

Bibliografia

- [1] PicoRV32 – A Size-Optimized RISC-V CPU. <https://github.com/YosysHQ/picorv32>
- [2] RISC-V GNU Compiler Toolchain. <https://github.com/riscv-collab/riscv-gnu-toolchain>
- [3] FIPS PUB 197, Advanced Encryption Standard (AES). <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.197.pdf>
- [4] Lattice ECP5 Family Data Sheet. <https://www.latticesemi.com/en/Products/FPGAandCPLD/ECP5>
- [5] An Efficient Hardware design and Implementation of Advanced Encryption Standard (AES) Algorithm. SINGH, K. P.; DOD, S. <https://eprint.iacr.org/2016/789.pdf>
- [6] Project Trellis: Documenting the Lattice ECP5 Architecture. <https://github.com/YosysHQ/prjtrellis>
- [7] Guia de Utilização da Colorlight i9 com Ferramentas OpenSource. AVELAR, J. N.; GOMES, G. P. UNICAMP, 2025.